Search Algorithm Comparison for the Wikipedia Navigation Problem

Nicholas Richard[1]

Department of Computer Science, University of New Brunswick,
Saint John, NB, Canada

Abstract. This project compares three search algorithms on the Wikipedia navigation problem, commonly known as Wikiracing: Iterative Deepening Depth-First Search (IDDFS), A* with a learned heuristic, and Bidirectional Breadth-First Search (BFS). The graph used is the SNAP English Wikipedia hyperlink dataset [1], containing over 4.2 million pages and 101 million directed edges. A* uses pretrained Wikipedia2Vec embeddings [7] combined with incoming link degree to estimate distance to the goal. Across six test pairs, all three algorithms consistently found optimal paths of the same length. Bidirectional BFS explored orders of magnitude fewer pages in every case, while A* consistently explored more pages than IDDFS despite its heuristic guidance. The results show that lightweight semantic and structural heuristics do not align well with shortest path distance in the Wikipedia link graph, and that exploiting graph structure directly is more effective than heuristic estimation in this domain.

Keywords: heuristic search, Wikipedia navigation, IDDFS, A* search, bidirectional BFS, graph search, representation learning

Table of Contents

Search Algorithm Comparison for the Wikipedia Navigation Problem 1
Nicholas Richard
1 Introduction 3
1.1 The Problem 3
1.2 Key Idea 3
1.3 Project Overview 3
2 Approach 3
2.1 Graph and Data Sources 3
2.2 Algorithms 4
2.3 Heuristic Design 5
3 Results 5
4 Discussion 6
4.1 Why Bidirectional BFS Dominates 6
4.2 On the Fairness of Bidirectional Search 6
4.3 Why A* Underperforms 7
4.4 Effect of the Link Cap 7
5 Implementation Revisions 8
6 Related Work 8
7 Conclusion 9
A Search Paths 10
B Source Code 10

1 Introduction

1.1 The Problem

Imagine you are on the Wikipedia page for *Cat* and your goal is to reach the page for *Football* by clicking hyperlinks, using only the links visible on each page. How few clicks does it take?

This is Wikiracing. Each Wikipedia article is a node in a directed graph and each hyperlink is an edge. The task is to find the shortest path between two nodes. What makes it an interesting computer science problem is the scale: over four million articles connected by more than a hundred million links. Brute-force search is slow, and finding a short path efficiently requires knowing something about how the graph is structured.

1.2 Key Idea

This project compares three strategies for solving the problem. IDDFS [3] uses no domain knowledge and searches paths in order of increasing length until a solution is found. A* [2] attempts to guide the search by scoring each candidate page based on its semantic similarity to the goal, using pretrained Wikipedia2Vec embeddings [7], and by preferring pages with high incoming link degree, on the hypothesis that well-connected hub pages tend to appear on short paths. Bidirectional BFS searches simultaneously from both the start and the goal, meeting in the middle, and exploits graph structure directly without any heuristic.

The central question is whether the semantic and structural signals A* uses provide useful guidance. Across three iterations of heuristic design, the results suggest they do not, and understanding why is the most instructive finding of the project.

1.3 Project Overview

All three algorithms are evaluated against the SNAP English Wikipedia hyperlink dataset [1], containing 4,198,894 pages and 101,311,613 directed edges. The implementation went through several iterations, including a switch from live API lookups to a preloaded graph, three rounds of heuristic design for A*, and the discovery that an earlier link cap substantially distorted the algorithm comparison. These decisions and their effects are discussed in Section 4.3 and Section 4.4. The rest of the paper describes the implementation in detail, presents results across six test queries, and analyzes the outcomes.

2 Approach

2.1 Graph and Data Sources

The SNAP English Wikipedia dataset [1] provides a snapshot of the Wikipedia hyperlink graph from 2013 as a directed edge list, paired with a title mapping

file. Both files are loaded at startup and stored in memory as Python dictionaries: a forward graph mapping each page to its outgoing links, and a reverse graph mapping each page to its incoming links. A normalized title index enables case-insensitive lookup so that user input does not need to match article casing exactly. The parsed graph is serialized to disk after the first load so that subsequent runs can skip the parsing step, reducing startup time from approximately one minute to a few seconds.

Page titles entered by the user are first matched against the SNAP title index directly. If no match is found, the MediaWiki API [5] is queried to resolve redirects and correct capitalization. This approach limits API calls to at most two per search, with the rest of the computation performed entirely in memory against the preloaded graph.

The initial implementation retrieved page links dynamically from the MediaWiki API during search. This approach was abandoned for two reasons. First, Wikipedia's API enforces rate limits that triggered 429 errors within seconds of a search starting, as all three algorithms make hundreds of requests in rapid succession. Second, even with deliberate delays between requests, wall-clock time was dominated by network latency rather than algorithmic behavior, making meaningful comparison impossible. Switching to the preloaded SNAP graph eliminated both problems and allowed the algorithms to be evaluated on their actual computational properties. The trade-off is that the graph reflects Wikipedia as it existed in 2013; pages created or links added since then are not represented. For the purposes of comparing algorithm behavior this is acceptable, as the graph remains large and well-connected, but paths found may differ from those a live implementation would produce.

2.2 Algorithms

Three search algorithms were implemented and compared.

IDDFS [3] performs depth-limited depth-first search with increasing depth limits, beginning at one and incrementing until a path is found or the maximum depth of six is reached. It guarantees the shortest path in unweighted graphs while keeping memory usage proportional to the depth of the solution rather than the size of the frontier. An explicit stack is used to avoid Python recursion limits. Cycle detection checks whether a neighbor already appears in the current path before expanding. The depth limit of six is imposed because IDDFS has $O(b^d)$ time complexity, where b is the branching factor and d is the depth. Without a limit the search becomes intractable for deep paths. In practice, six is sufficient: the Wikipedia graph is highly connected and the vast majority of page pairs are reachable within this depth, as confirmed by all six test cases in this evaluation.

A* maintains a priority queue (min-heap) ordered by $f(n) = g(n) + h(n)$, where $g(n)$ is the number of links traversed and $h(n)$ is the heuristic estimate. Parent pointers are used to reconstruct the path upon termination rather than storing the full path in each heap entry, reducing memory overhead. A cost dictionary tracks the best known distance to each page and prevents re-expansion via suboptimal routes.

Bidirectional BFS searches simultaneously from the start and the goal, alternating between expanding the smaller frontier. The forward search uses outgoing links and the backward search uses incoming links from the reverse graph. When a page appears in both frontiers, the two half-paths are joined and the search terminates. This reduces the theoretical search space from $O(b^d)$ to $O(b^{d/2})$, where b is the branching factor and d is the path depth.

2.3 Heuristic Design

The A* heuristic combines two signals. The semantic component uses cosine similarity between Wikipedia2Vec entity vectors [7], which are pretrained on Wikipedia's text and link structure. The goal page's embedding is retrieved once before search begins. For each candidate page, cosine similarity to the goal vector is computed and converted to a cost in $[0, 1]$, where 0 indicates identical embeddings and 1 indicates orthogonal or missing vectors. The structural component normalizes the incoming link count for a page by the maximum incoming degree across the graph, producing a score in $[0, 1]$ where pages with high incoming degree score lower. The two components are combined as:

$$h(n) = 0.7 \times semantic_score(n) + 0.3 \times degree_score(n) \tag{1}$$

Semantic similarity is weighted more heavily because it is goal-directed, while degree is goal-agnostic. The heuristic is computed entirely from preloaded data and requires no API calls during search. Both embedding lookups and heuristic scores are cached per run to avoid redundant computation.

The heuristic went through three iterations. The first used pure Wiki2Vec cosine similarity. The second added outgoing link degree as a structural signal. The third and final version replaced outgoing degree with incoming degree, on the hypothesis that pages many articles link to are more likely to appear on shortest paths between arbitrary pairs. Results across all three iterations were nearly identical, which is itself a finding discussed in Section 4.3.

3 Results

Six query pairs were evaluated, selected to vary in topic distance and path depth. All three algorithms found paths of the same length in every case, confirming that the Wikipedia graph is well-connected enough for short paths to exist between arbitrary topic pairs within a depth of six. Throughout, *pages expanded* refers to pages whose link lists were actually fetched and examined by the algorithm, not pages that merely appeared as a hyperlink on another page. A page is counted only when the algorithm takes it from the frontier and inspects its neighbors. The results are summarized in Table 1.

Table 1. Pages expanded and wall-clock time per algorithm across six query pairs. All algorithms found paths of equal length.

Query	IDDFS	A*	BiDir	Path len.
Cream → HOSE	226,961	897,194	63	5
Drooling → Anglerfish	1,367	3,374	8	3
Window → Nigeria	154	11,367	3	3
Napkin → Pepsi	27	2,501	3	3
Dependency → Guitar pick	17,391	49,407	40	4
Abacus → Thermodynamics	915	12,316	4	3

Bidirectional BFS explored dramatically fewer pages than both IDDFS and A* across all queries. In every test case it outperformed both algorithms by several orders of magnitude, often exploring fewer than 65 pages compared to hundreds of thousands for IDDFS and nearly a million for A* in the worst case. In the most extreme case (Cream to HOSE, path length 5), Bidirectional explored 63 pages while IDDFS explored 226,961 and A* explored 897,194.

IDDFS performed well relative to its uninformed nature, particularly on shorter paths. For Napkin to Pepsi (path length 3), IDDFS explored only 27 pages by committing to the first viable DFS branch rather than scoring all neighbors.

A* consistently explored more pages than IDDFS. On Window to Nigeria, A* explored 11,367 pages against IDDFS's 154. On Dependency to Guitar pick, A* explored 49,407 pages against IDDFS's 17,391. The gap widened on longer paths where the heuristic's inability to guide search effectively compounded over more expansion steps.

4 Discussion

4.1 Why Bidirectional BFS Dominates

Bidirectional BFS's advantage is structural. By searching from both ends simultaneously and using the reverse graph for backward expansion, it meets in the middle after each frontier grows to roughly $O(b^{d/2})$ pages rather than $O(b^d)$. For a path of length 5 with a branching factor in the hundreds, this is a difference of several orders of magnitude. The algorithm requires no heuristic and makes no assumptions about the graph beyond having access to incoming links, which the preloaded reverse graph provides.

4.2 On the Fairness of Bidirectional Search

Bidirectional BFS gains its advantage by searching simultaneously from both the start and the goal, using a precomputed reverse graph to follow incoming links during backward expansion. It is reasonable to ask whether this constitutes an unfair advantage relative to the other algorithms or relative to the game itself.

On the algorithmic side, the reverse graph contains no information beyond what is already present in the forward graph. Every incoming link for a page is implied by the corresponding outgoing link elsewhere. Precomputing the reverse index at load time is an implementation detail, not a source of privileged information. Any algorithm could derive the same data by scanning the forward graph, at the cost of an $O(n)$ lookup per page rather than $O(1)$.

On the game side, the Wikipedia navigation game as described in [6] defines the objective as navigating from a source article to a target article by following hyperlinks, with no explicit constraint on search direction or strategy. The restriction on following links forward applies to human players clicking through a browser interface, not to algorithmic traversal of a graph. Since bidirectional search is not prohibited by the game's rules, and since the reverse graph introduces no external information, Bidirectional BFS is treated as fair in this evaluation. Its performance advantage reflects a genuine algorithmic property, specifically its $O(b^{d/2})$ complexity, rather than access to privileged data.

4.3 Why A* Underperforms

A*'s poor performance relative to IDDFS comes down to two factors. First, heuristic evaluation is expensive relative to the cost of expansion. For each page expanded, A* scores all neighbors before pushing them to the heap. With hundreds of neighbors per page, this means many embedding lookups and cosine similarity computations per expansion step, even with caching. IDDFS incurs no such overhead.

Second, and more importantly, the heuristic does not provide useful directional guidance. Semantic similarity between Wikipedia2Vec embeddings reflects topical relatedness in text and link co-occurrence, not navigational proximity in the link graph. Two pages that are semantically close may be many links apart, and two pages with no obvious semantic relationship may be directly linked. The three heuristic iterations (semantic only, semantic plus outgoing degree, semantic plus incoming degree) produced nearly identical exploration counts, suggesting that neither signal carries meaningful information about graph distance in this domain. As a result, A* behaves like an expensive version of uninformed search, incurring additional computation without gaining meaningful guidance.

West and Leskovec [4] found a related pattern in human Wikiracing: successful players initially route through high-degree hub pages to close the distance, before switching to content-similarity cues to home in on the target. The structure of the Wikipedia graph favors strategies that route through hubs early, which is what Bidirectional BFS discovers organically.

4.4 Effect of the Link Cap

An earlier version of the implementation capped links per page at 50 to limit API request volume. Under this cap, A* appeared competitive with IDDFS because IDDFS's DFS traversal was exploring artificially pruned branches, causing it to miss short paths and explore more nodes to compensate. Once the cap was

removed and the full SNAP graph used, IDDFS's true behavior emerged: it commits quickly to a viable path and terminates with far less exploration than A*. This reversal shows how implementation constraints can substantially distort algorithm comparisons, and highlights the importance of evaluating algorithms on the full problem rather than a simplified proxy.

5 Implementation Revisions

Several revisions were made during development as problems were discovered in testing. The details of each are discussed where relevant in the sections above; this section lists them together for clarity.

Live API to preloaded graph. The initial implementation fetched page links from the MediaWiki API during search. Rate limiting made this impractical, and network latency dominated wall-clock time, making algorithm comparison meaningless. The implementation was switched to the preloaded SNAP graph.

Link cap removed. An early version capped outgoing links per page at 50 to reduce API request volume. When the SNAP graph was adopted this cap was retained, which artificially constrained IDDFS and distorted the comparison. Removing it revealed IDDFS's true performance.

Heuristic: semantic only to semantic plus outgoing degree. The first heuristic used pure Wikipedia2Vec cosine similarity. Outgoing link degree was added as a structural signal on the hypothesis that pages with many outgoing links are more likely to sit near short paths. Results were nearly identical.

Heuristic: outgoing degree to incoming degree. Outgoing degree was replaced with incoming degree, on the hypothesis that pages many articles link to act as navigational hubs more likely to appear on shortest paths. Again, results were nearly identical across all test cases, which itself became a finding.

6 Related Work

A* was introduced by Hart, Nilsson, and Raphael [2] and remains a standard reference for heuristic search. Its optimality guarantee requires an admissible heuristic. Without admissibility, A* may find paths longer than optimal but can still terminate faster than uninformed search when the heuristic is well-matched to the domain. In this project the heuristic is not admissible, and A* does not terminate faster.

Korf [3] introduced iterative deepening depth-first search as an algorithm that achieves the optimality of BFS with the memory efficiency of DFS, showing that the overhead of re-exploration is small in practice because most nodes reside at the deepest level.

West and Leskovec [4] studied human navigation on Wikipedia and found that successful players use hub-centric strategies in the early phase of navigation, before switching to semantic cues. This is consistent with the finding here that

semantic heuristics alone do not effectively guide algorithmic search on the same graph.

The SNAP dataset [1] provides the graph used throughout this project. Yamada et al. [7] describe the Wikipedia2Vec model used for entity embeddings.

7 Conclusion

Three search algorithms were implemented and compared on the Wikipedia navigation problem using the SNAP English Wikipedia graph. All three found optimal paths across six query pairs. Bidirectional BFS explored orders of magnitude fewer pages in every case, demonstrating the practical advantage of its $O(b^{d/2})$ complexity. IDDFS performed efficiently by committing to paths early. A*, despite three iterations of heuristic design combining Wikipedia2Vec semantic similarity and link degree, explored more pages than IDDFS in every test.

The main finding is that lightweight heuristics based on semantic similarity and graph degree are poor proxies for shortest path distance in the Wikipedia link graph. The graph's structure is too irregular and topic-agnostic for these signals to provide useful guidance. More effective approaches would require precomputed distance estimates, such as landmark-based heuristics, which were outside the scope of this project. The link cap experiment also shows that implementation constraints can substantially distort algorithm comparisons, and that fair evaluation requires running algorithms on the full problem. These results suggest that in large real-world graphs like Wikipedia, exploiting structural properties directly is more effective than relying on lightweight heuristic estimation.

References

1. Boldi, P., Vigna, S.: The WebGraph framework I: Compression techniques. In: Proceedings of the 13th International World Wide Web Conference (WWW 2004). pp. 595–601. ACM Press (2004), dataset: `https://snap.stanford.edu/data/enwiki-2013.html`
2. Hart, P.E., Nilsson, N.J., Raphael, B.: A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics **4**(2), 100–107 (1968)
3. Korf, R.E.: Depth-first iterative-deepening: An optimal admissible tree search. Artificial Intelligence **27**(1), 97–109 (1985)
4. West, R., Leskovec, J.: Human wayfinding in information networks. In: Proceedings of the 21st International World Wide Web Conference (WWW). pp. 619–628 (2012)
5. Wikimedia Foundation: MediaWiki API documentation. `https://www.mediawiki.org/wiki/API:Main_page` (2024)
6. Wikipedia contributors: Wikiracing. `https://en.wikipedia.org/wiki/Wikiracing` (2024)
7. Yamada, I., Asai, A., Sakuma, J., Shindo, H., Takeda, H., Takefuji, Y., Matsumoto, Y.: Wikipedia2Vec: An efficient toolkit for learning and visualizing the embeddings of words and entities from Wikipedia. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. pp. 23–30. Association for Computational Linguistics (2020)

A Search Paths

The following are the paths found by each algorithm across the six test queries. Paths may differ between algorithms while remaining the same length, as multiple shortest paths typically exist in a graph of this density.

Cream → HOSE

- IDDFS: Cream → Human factors and ergonomics → Missouri → Missouri University of Science and Technology → World Series of Poker → HOSE
- A*: Cream → Engineering → Outline of software engineering → Poker → Texas hold 'em → HOSE
- BiDir: Cream → Engineering → University of Minnesota → University of Minnesota Law School → World Series of Poker → HOSE

Drooling → Anglerfish

- IDDFS: Drooling → Lung → Actinopterygii → Anglerfish
- A*: Drooling → Insect → Bioluminescence → Anglerfish
- BiDir: Drooling → Lung → Actinopterygii → Anglerfish

Window → Nigeria

- IDDFS: Window → Wall → Terracotta → Nigeria
- A*: Window → German language → Cameroon → Nigeria
- BiDir: Window → Door → Natural rubber → Nigeria

Napkin → Pepsi

- IDDFS: Napkin → Ancient Roman cuisine → Salt → Pepsi
- A*: Napkin → South Africa → Coca-Cola → Pepsi
- BiDir: Napkin → Ancient Roman cuisine → Salt → Pepsi

Dependency → Guitar pick

- IDDFS: Dependency → Substance dependence → Lysergic acid diethylamide → Paul McCartney → Guitar pick
- A*: Dependency → Dependent territory → Madeira → Ukulele → Guitar pick
- BiDir: Dependency → Substance dependence → Lysergic acid diethylamide → Paul McCartney → Guitar pick

Abacus → Thermodynamics

- IDDFS: Abacus → Calculator → Differential equation → Thermodynamics
- A*: Abacus → Addition → Classical mechanics → Thermodynamics
- BiDir: Abacus → Jean-Victor Poncelet → France → Thermodynamics

B Source Code

The full implementation is available at:

`https://github.com/nrichar1/wiki-navigation-ai`

The repository includes the main search implementation, dataset download instructions, and the Wikipedia2Vec model download link.